The University Of Sheffield.

General Engineering

Interim Report Assignment Cover Sheet

Surname: McCleary Forenames: Patrick John

Registration number 200346940

Module code: ELE418

Module name: Individual Advanced Project

Assignment title: The application of regularisation to common datasets in machine learning

Supervisor: Dr J R Winkler

Date submitted: 14/12/2025

Student Declaration

I confirm that the work I have submitted is my own original work and that I have made appropriate references to any sources used.
I agree to any submitted materials being used by the Department and/or University to detect and take action on any unfair means.
I confirm that the printed version of this assessment and any electronic copy I am required to submit to TurnItIn are identical.
I understand that failure to meet these guidelines may result in one or more of the following penalties being imposed:

1. A reduced mark (including a mark of zero) being recorded for this work.
2. A reduced mark (including a mark of zero) being recorded for the unit/module.
3. A formal reprimand (warning) being placed on my student record at Departmental and/or University level which may be taken into account whenever references are written.
4. Reporting of the incident to the University Disciplinary Committee.
5. Suspension or expulsion from the University.

Student's signature: PMcC Date: 14/12/2025

MEng General Engineering Project

Interim Report

The application of regularisation to common datasets in machine learning

Patrick J McCleary

December 14, 2025

Supervisor: Dr J R Winkler

Contents

1 Introduction

Regularisation is a process which is often applied when data are analysed to impose stability and robustness on the model. This should be applied to problems that are either highly unstable, meaning that the output is extremely sensitive to changes or 'noise' in the input, or where no unique solution exists – these problems are described as being ill-posed. It does this by solving a 'neighbouring' well-conditioned problem, which inherently introduces some amount of error therefore regularisation can be thought of as deliberately trading a small amount of error for improved stability.

However, the problem is that the use of regularisation is often blindly applied and rarely justified which is an issue because its use is not benign meaning applying it unnecessarily causes harm as it can actually produce an incorrect output if applied when not needed.

The purpose of this interim report is to build a strong theoretical foundation to determine when regularisation is required by reviewing key concepts including ill-posedness, mathematical theory such as singular value decomposition (SVD) of a matrix and the criteria for determining its necessity. These concepts form the basis for the analysis to be conducted in the final report where they will be applied to common publicly available machine learning datasets to prove what is suspected – that regularisation is often applied unnecessarily.

2 Aims and Objectives

The primary aim of this project is to define the mathematical conditions which determine if regularisation is needed. To do this, a software environment must be set up to implement and test these criteria. This was done in MATLAB because of the clearer documentation, easier handling of linear algebra and the availability of the "Regularization Tools" package (Hansen 1994). The next step commencing in semester two is to choose publicly available datasets or problems relevant to machine learning. These criteria and MATLAB framework will be used to rigorously test the datasets and verify that the regularised solution satisfies conditions regarding stability and error.

3 Literature Review

3.1 Mathematical Foundations

Inverse problems in linear algebra, where observed data are used to determine the model parameters, are fundamental to machine learning. Key applications include image deblurring, inverse kinematics of a robot arm and a control system attempting to balance two stacked spheres. These problems can be represented by a system of linear equations of the form

$$\mathbf{Ax} = \mathbf{b} \tag{1}$$

where $\mathbf{A}$ represents the system, $\mathbf{x}$ represents the model parameters and $\mathbf{b}$ represents the observed data.

Inverse problems are prone to ill-posedness, a concept first defined at the beginning of the 20th century (Hadamard 1902) as failing to meet any of the following criteria: the solution must exist, the solution must be unique and the solution must depend continuously on the input. It is this third qualifier, the stability condition, that inverse problems most commonly fail since the processes they reverse are smoothing or diffusive with a prime example being deblurring. This instability of these problems necessitates a mathematical diagnosis which the SVD addresses.

The SVD is a fundamental tool in regularisation because it reveals the quantitative measure of the ill-posedness of a problem. As its name suggests, the SVD factorises the matrix $\mathbf{A}$ into three simpler matrices: two orthogonal matrices and a diagonal matrix $\mathbf{\Sigma}$ with singular values.

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{2}$$

The matrix $\mathbf{A}$ is said to be ill-conditioned if the ratio $\sigma_{max}/\sigma_{min}$, known as the condition number, is large (Turing 1948). This is because as $\sigma_i \to 0$, the term $1/\sigma_i$ becomes arbitrarily large, which amplifies the noise in the input and renders the direct inversion unstable.

The most common regularisation method used is Tikhonov regularisation (Tikhonov 1963) which can be elegantly described with a single equation:

$$\mathbf{x}_\lambda = \underset{\mathbf{x}}{\operatorname{argmin}} \left(\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 + \lambda^2 \|\mathbf{x}\|_2^2 \right) \tag{3}$$

The significance of the regularisation parameter λ is that it controls the trade-off between the stability and accuracy of the solution. It essentially acts as a spectral filter, suppressing the contributions from small singular values where noise amplification is most severe, while preserving the information contained in the dominant singular values. Therefore, it is clear that the choice of this parameter is crucial, for which there are two different approaches: numerical and statistical.

3.2 Numerical Approaches

The L-curve is a numerical method for determining the optimum regularisation parameter λ which consists of a parametric plot of the solution norm $\|\mathbf{x}_\lambda\|_2$ against the residual norm $\|\mathbf{A}\mathbf{x}_\lambda - \mathbf{b}\|_2$ on log-log axes as the regularisation parameter λ varies. This was first used by Lawson & Hanson (1974) and further developed by Hansen & O'Leary (1993) where it was formally introduced. The L-curve serves two purposes: to identify by visual inspection (namely the presence of a distinct corner) whether regularisation is required and to determine the regularisation parameter λ which was explored further in Section 4.1. Therefore, the L-curve is a critical diagnostic tool for this project, allowing the determination of the genuine need for regularisation in machine learning datasets.

Analogous to how the L-curve provides a visual indication of ill-posedness, the Discrete Picard Condition (DPC) is a way to determine quantitatively if a stable solution is attainable. The DPC is concerned with the behaviour of the Fourier coefficients $c_i = \mathbf{u}_i^T \mathbf{b}$ relative to the singular values σ_i. This concept was first introduced by Varah (1979) who observed that the Fourier coefficients must decay to zero faster than the singular values for a solution to be stable — this was later formally defined by Hansen (1990) as the DPC. The necessity for the DPC is evident in the expansion of the unregularised solution:

$$\mathbf{x}_{LSQ} = \sum_{i=1}^{n} \frac{c_i}{\sigma_i} \mathbf{v}_i \tag{4}$$

If the numerator $|c_i|$ does not decay faster than the denominator σ_i, the individual terms in the sum will diverge, resulting in a solution dominated by noise. The DPC, which plots the magnitude of Fourier coefficients, singular values and the decay rate $|c_i|/\sigma_i$ against the indices, was implemented and tested in Section 4.2.

3.3 Statistical Approach

Generalised Cross-Validation (GCV) is a statistical method for choosing the regularisation parameter in Tikhonov regularisation (3). Crucially, it provides an estimate without requiring prior knowledge of the noise variance (Golub et al. 1979) unlike the Discrepancy Principle which relies on an accurate estimate of the noise. This builds on the statistical concept of cross-validation first introduced by Stone (1974) as a way to measure prediction error by leaving out some data, fitting the model, and assessing the ability of the model to predict the missing data. Instead of leaving out data, GCV relies upon linear algebra to efficiently approximate the prediction error. Specifically, the chosen regularisation parameter is the one that minimises the following function:

$$G(\lambda) = \frac{\|\mathbf{A}\mathbf{x}_\lambda - \mathbf{b}\|_2^2}{\left(m - \sum_{i=1}^{n} \frac{\sigma_i^2}{\sigma_i^2 + \lambda^2}\right)^2} \tag{5}$$

where m represents the number of data points (the rows of matrix $\mathbf{A}$). The function $G(\lambda)$ typically forms a convex curve when plotted against λ, which allows the optimal λ to be identified at the minimum. This statistical method was implemented and evaluated using MATLAB as detailed in Section 4.3.

4 Current Status of Work

A software environment was setup with MATLAB using version 4.1 of the "Regularization Tools" package. Since the common machine learning datasets had not yet been chosen, these initial experiments were applied to the `shaw` test problem which is a standard "one-dimensional image restoration model" (Hansen 1994). This is a discretisation of a Fredholm integral equation of the first kind which is a severely ill-posed problem. The dimensions of the forward process matrix $\mathbf{A}$ were set to 50×50.

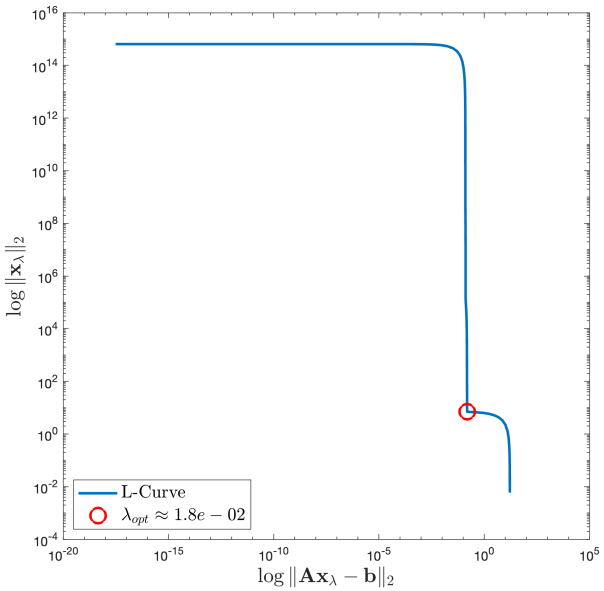
4.1 Verification of L-curve

The aim of the first experiment was to manually generate the L-curve and curvature plot for the test problem described. Gaussian white noise with a noise level of 1% was added to the true observed data vector $\mathbf{b}$. This was done with a fixed random seed to ensure the reproducibility of the experiment. To avoid the computational expense of inverting matrix $\mathbf{A}$ for every discrete λ point, the SVD was computed once. The number of discrete points was set to 200, logarithmically spaced between $0.1\sigma_{\min}^2$ and $10\sigma_{\max}^2$. At each λ, the Tikhonov filter factor was computed to calculate the solution norm $\eta = \|\mathbf{x}_\lambda\|_2$ and the residual norm $\rho = \|\mathbf{A}\mathbf{x}_\lambda - \mathbf{b}\|_2$ which when plotted form the L-curve.

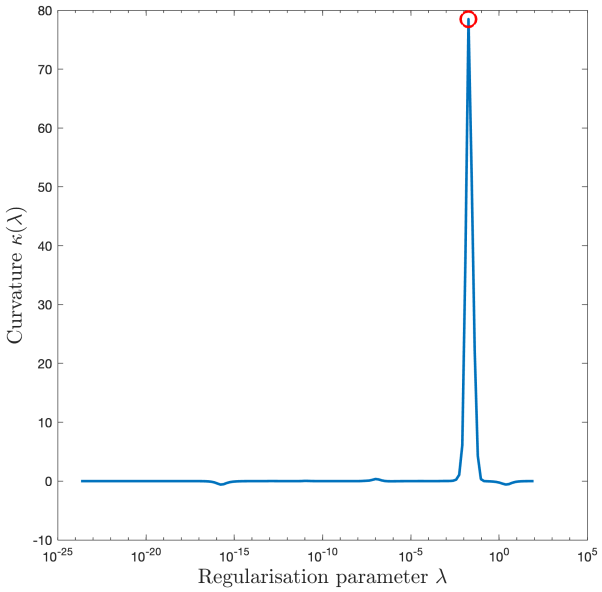
The curvature κ was calculated using a discrete finite-difference approximation applied to the standard planar curve formula which was used by Hansen & O'Leary (1993):

$$\kappa(\lambda) = \frac{\hat{\rho}'\hat{\eta}'' - \hat{\rho}''\hat{\eta}'}{((\hat{\rho}')^2 + (\hat{\eta}')^2)^{3/2}} \tag{6}$$

Note that in (6), $\hat{\rho} = \log \rho$ and $\hat{\eta} = \log \eta$. This differs from the more robust spline method used in `l_corner`. This method, described above with the code shown in Appendix B, yielded the following results.



(a) Manual L-curve construction.



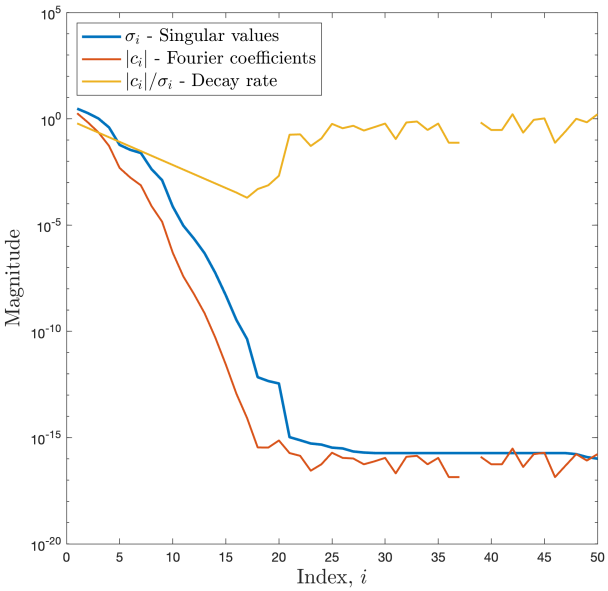
(b) Variation of κ with λ.

Figure 1: Experimental verification of the L-curve criterion on the `shaw` problem. (a) The L-curve with the identified corner. (b) The corresponding curvature peak used to identify λ_{opt}.

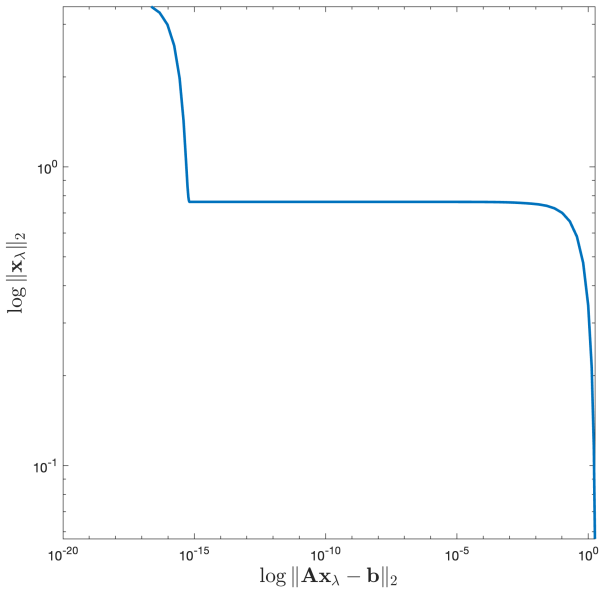
The results of this verification shown in Figure 1 capture the behaviour of regularisation. The horizontal plateau in Figure 1a represents the asymptotic convergence to the naive solution found with the pseudoinverse $\mathbf{x}_{naive} = \mathbf{A}^\dagger\mathbf{b}$. In this region, λ is insufficient to filter out noise, resulting in overfitting (low bias, high variance) where the model fits the noise rather than the signal. Increasing λ further leads to the distinct corner which is the optimal trade-off — this corresponds to the position of maximum curvature in Figure 1b identifying λ_{opt}. Increasing λ beyond λ_{opt} results in underfitting (high bias, low variance) where the solution becomes over-smoothed, causing a rapid increase in residual error.

4.2 Verification of DPC

Two different scenarios were tested with an experiment: The aim was to demonstrate the Picard plots and corresponding L-curves of problems which obey and violate the DPC. This experiment was applied to the same `shaw` test problem defined in Section 4. For DPC satisfied, a clean observed data vector $\mathbf{b}$ was constructed to ensure it obeyed the DPC by setting $c_i = \sigma_i e^{-\alpha i}$, representing exponential decay with a decay constant of $\alpha = 0.5$. The observed data vector $\mathbf{b}$ was then reconstructed with $\mathbf{b} = \mathbf{U}\mathbf{c}$. For DPC not satisfied, the observed data vector $\mathbf{b}$ was generated using pure white noise (random Gaussian distribution). The number of discrete points was set to 100, logarithmically spaced between $0.1\sigma_{\min}^2$ and $10\sigma_{\max}^2$. At each point the solution η and ρ residual norms were computed and plotted. Also, the magnitudes of the Fourier coefficients $|c_i|$, singular values σ_i and decay rate $|c_i|/\sigma_i$ were plotted against the index. The MATLAB code for this experiment can be found in Appendix C.



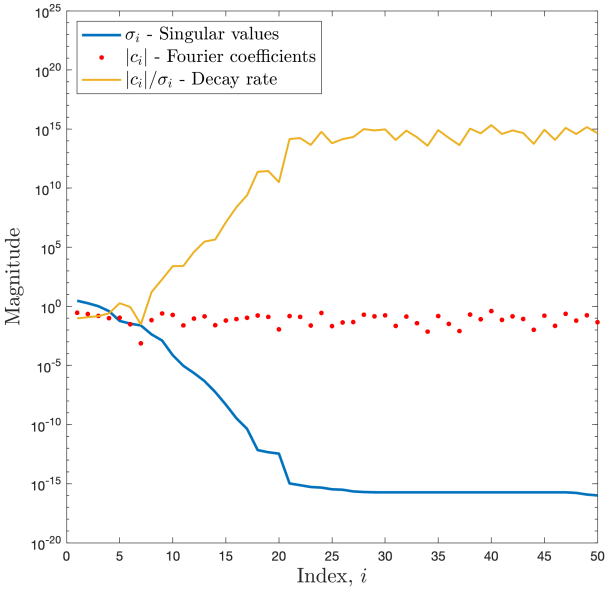
(a) Picard plot for DPC satisfied.



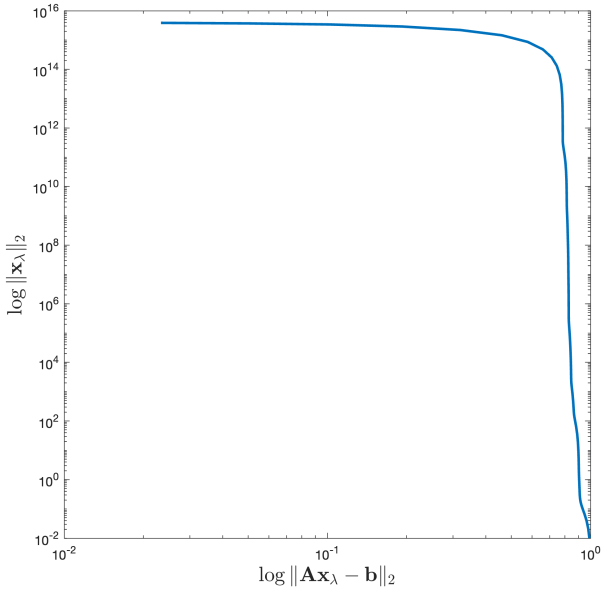
(b) L-curve for DPC satisfied.

Figure 2: Experimental verification of the DPC for decay constant $\alpha = 0.5$. (a) The Picard plot showing the rate of decay. (b) The L-curve with the distinct corner.

The results of this experiment, shown in Figure 2, verify the DPC. This is because Figure 2a shows that the Fourier coefficients decay to zero faster than the singular values which means the DPC is satisfied. The corresponding L-curve shown in Figure 2b has the distinct corner and L shape which indicates that regularisation is necessary.



(a) Picard plot for DPC not satisfied.



(b) L-curve for DPC not satisfied.

Figure 3: Experimental verification of the DPC. (a) The Picard plot showing the rate of decay. (b) The L-curve without the distinct corner.

Figure 3 shows the Picard plot and associated L-curve for a problem where the DPC is not satisfied. It is clear the DPC is not satisfied because the Fourier coefficients fail to decay to zero faster than the singular values as shown in Figure 3a. The L-curve in Figure 3b lacks the

distinct corner and L shape which indicates that regularisation must not be applied and the regularisation parameter must be set to $\lambda = 0$. The L-curve shows that there is no value of λ which minimises both the solution and residual norm simultaneously.

4.3 Verification of GCV

The aim of this experiment was to verify the GCV using the same test problem, noise level and fixed random seed as in Section 4.1. The code, shown in Appendix E, uses (5) to plot the function and finds the minimum point to locate the optimum regularisation parameter λ, shown in Appendix D.

The results in Figure 5 yield a regularisation parameter of $\lambda_{GCV} \approx 4.1 \times 10^{-2}$. This is consistent in order of magnitude with the value obtained with the L-curve ($\lambda_{opt} \approx 1.8 \times 10^{-2}$), which verifies the method as a viable statistical approach. However, the L-curve criterion is often preferred since the corner in the L-curve is geometrically distinct, whereas the GCV function exhibits a shallow minimum as seen in Figure 5, making λ identification more sensitive to errors.

5 Self-Review

While the plan outlined in the aims and objectives submitted at the end of week 4 has changed, all tasks I was challenged with over the course of the semester were completed successfully and I took the time to properly understand the mathematical theory behind the code I was writing and do further reading to supplement my learning.

The project up to this point has been a steep learning curve due to the very technical nature of the work and my lack of experience and knowledge of computer science concepts, machine learning and programming. Though my mechanical engineering background gave me a strong mathematical foundation to build upon, linear algebra concepts such as SVD and pseudoinverse were completely new to me. However I believe my experience of learning difficult concepts such as fluid dynamics has prepared me well. I found using MATLAB to be very intuitive and that it was better to manually derive solutions – as it turned out, translating the mathematical derivations into code proved straightforward.

I think the original plan may have been a little optimistic as the plan was to choose the datasets this semester, however I believe prioritising a comprehensive theoretical foundation and verified software environment in the first semester will place the project on a stronger footing to apply this theory.

6 Project Management and Planning

The main goal for semester two is to put this strong foundation of mathematical theory and software environment into practice with extensive testing of the chosen machine learning datasets. With the datasets the scope for testing is broad, with initial ideas for investigation including a comparison of different methods of parameter-choice, different regularisation methods (Lasso vs Ridge vs Elastic net), and sensitivity studies on ill-posedness severity and noise levels. The direction taken will be refined following feedback received on this report, so it is important to have a plan that is highly flexible. Since the final report will expand extensively on the interim report, more time has been allocated to report writing. Lessons learned from this semester have been applied to the updated Gantt chart (Figure 4) which include adding more contingency, accounting for coursework deadlines in other modules and adding rehearsal time for oral presentation.

7 Bibliography

Golub, G. H., Heath, M. & Wahba, G. (1979), 'Generalized cross-validation as a method for choosing a good ridge parameter', *Technometrics* **21**, 215–223.

Hadamard, J. (1902), 'Sur les problèmes aux dérivés partielles et leur signification physique', *Princeton University Bulletin* **13**, 49–52.

Hansen, P. C. (1990), 'The discrete picard condition for discrete ill-posed problems', *BIT Numerical Mathematics* **30**(4), 658–672.

Hansen, P. C. (1994), 'Regularization tools: A Matlab package for analysis and solution of discrete ill-posed problems', *Numerical Algorithms* **6**(1), 1–35.

Hansen, P. C. & O'Leary, D. P. (1993), 'The use of the L-Curve in the regularization of discrete ill-posed problems', *SIAM Journal on Scientific Computing* **14**, 1487–1503.

Lawson, C. L. & Hanson, R. J. (1974), *Solving Least Squares Problems*, Prentice-Hall, Englewood Cliffs, NJ.

Stone, M. (1974), 'Cross-validatory choice and assessment of statistical predictions', *Journal of the royal statistical society: Series B (Methodological)* **36**(2), 111–133.

Tikhonov, A. N. (1963), 'Solution of incorrectly formulated problems and the regularization method', *Soviet Mathematics Doklady* **4**, 1035–1038.

Turing, A. M. (1948), 'Rounding-off errors in matrix processes', *The Quarterly Journal of Mechanics and Applied Mathematics* **1**(1), 287–308.

Varah, J. (1979), 'A practical examination of some numerical methods for linear discrete ill-posed problems', *Siam Review* **21**(1), 100–111.

A Gantt Chart

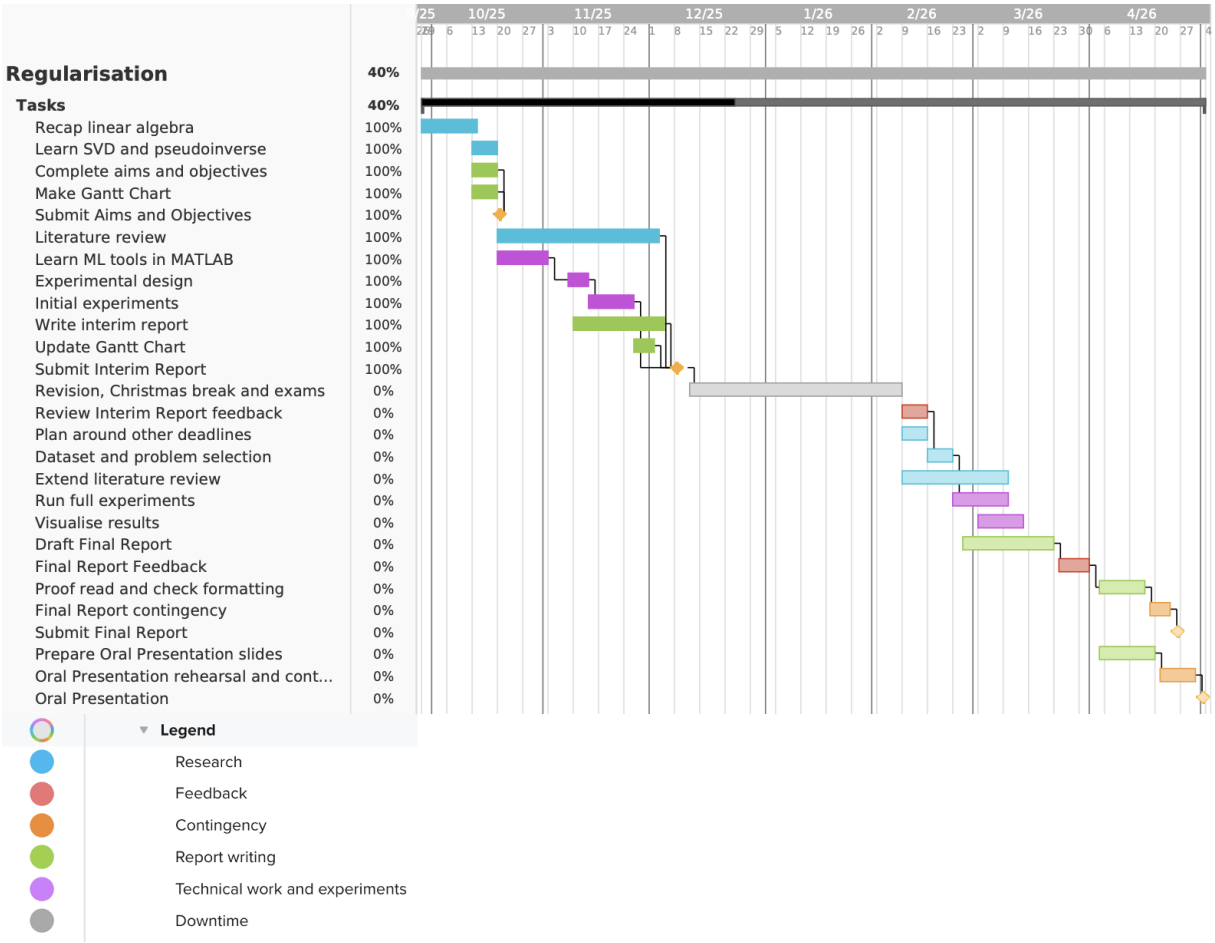


Figure 4: Updated Gantt Chart for the remainder of the project.

B MATLAB code for L-curve verification

The following MATLAB script was developed to manually generate the L-curve and mathematically verify the corner location using finite difference curvature approximation, as discussed in Section 4.1.

```
% Setup Shaw Problem with 1% Noise
n = 50;
[A, b_true, x_true] = shaw(n);

noise_level = 0.01;
randn('state', 0);
e = randn(n, 1);
e = e / norm(e) * noise_level * norm(b_true);
b = b_true + e;

% Compute SVD
[U, s, V] = csvd(A);
beta = U'*b;

% Define Lambda Range
lambda_start = 0.1 * s(end)^2;
lambda_end = 10 * s(1)^2;
lambdas = logspace(log10(lambda_start), log10(lambda_end), 200);

n_lambda = length(lambdas);
eta = zeros(n_lambda, 1);
rho = zeros(n_lambda, 1);

% Calculate Solution and Residual Norms
for i = 1:n_lambda
    lam = lambdas(i);
    f = s.^2 ./ (s.^2 + lam^2);
    rho(i) = norm( (1-f) .* beta );
    eta(i) = norm( (f .* beta) ./ s );
end

% Calculate Curvature (Finite Difference)
rho_hat = log(rho);
eta_hat = log(eta);

x_prime = gradient(rho_hat);
y_prime = gradient(eta_hat);
x_double_prime = gradient(x_prime);
y_double_prime = gradient(y_prime);

kappa = (x_prime .* y_double_prime - x_double_prime .* y_prime) ./ ...
        ((x_prime.^2 + y_prime.^2).^1.5);

% Find Maximum Curvature
[max_k, idx_max] = max(kappa);
opt_lambda = lambdas(idx_max);

% Plotting
figure('Visible', 'on');
loglog(rho, eta, '-', 'LineWidth', 2); hold on;
loglog(rho(idx_max), eta(idx_max), 'ro', 'MarkerSize', 10, 'LineWidth', 2);
title('L-curve', 'Interpreter', 'latex', 'FontSize', 14);
xlabel('$\log \| \mathbf{A}\mathbf{x}_{\lambda} - \mathbf{b} \|_2$', '
    Interpreter', 'latex', 'FontSize', 12);
ylabel('$\log \| \mathbf{x}_{\lambda} \|_2$', 'Interpreter', 'latex', 'FontSize'
    , 12);
```

```
legend('L-curve', ['$\lambda_{opt} \approx ' num2str(opt_lambda, '%.1e') '$'], ...
        'Location', 'SouthWest', 'Interpreter', 'latex');
grid on;
saveas(gcf, 'lcurve.png');

figure('Visible', 'on');
semilogx(lambdas, kappa, '-', 'LineWidth', 2); hold on;
semilogx(opt_lambda, max_k, 'ro', 'MarkerSize', 10, 'LineWidth', 2);
title('Variation of curvature $\kappa$ with regularisation parameter $\lambda$', ...
      'Interpreter', 'latex', 'FontSize', 14);
xlabel('Regularisation parameter $\lambda$', 'Interpreter', 'latex', 'FontSize', ...
      12);
ylabel('Curvature $\kappa(\lambda)$', 'Interpreter', 'latex', 'FontSize', 12);
grid on;
saveas(gcf, 'curvature.png');
```

Listing 1: MATLAB script for manual L-curve analysis on the `shaw` problem.

C MATLAB code for DPC verification

The following MATLAB script was developed to manually generate the Picard plot and associated L-curve as discussed in Section 4.2.

```
% Problem Definition
n = 50;
A = shaw(n);

% Compute SVD
[U, s, V] = csvd(A);
indices = (1:n)';

y_min_satisfied = 1e-20;
y_max_satisfied = 1e5;

y_min_unsatisfied = 1e-20;
y_max_unsatisfied = 1e25;

% Define Lambda Range
lambda_start = 0.1 * s(end)^2;
lambda_end = 10 * s(1)^2;
lambdas = logspace(log10(lambda_start), log10(lambda_end), 100);

% DPC Satisfied
alpha = 0.5;
c_clean = s .* exp(-alpha * indices);
b = U * c_clean;
c = U' * b;
ratio = abs(c) ./ s;
rho = zeros(1, 100);
eta = zeros(1, 100);
for k = 1:100
    lambda = lambdas(k);
    f = s.^2 ./ (s.^2 + lambda);
    rho(k) = norm( (lambda ./ (s.^2 + lambda)) .* c );
    eta(k) = norm( (f .* c) ./ s );
end

% Plotting
figure('Visible', 'on', 'Units', 'pixels', 'Position', [100 100 600 500]);
semilogy(indices, s, '-', 'LineWidth', 2); hold on;
semilogy(indices, abs(c), '-', 'LineWidth', 1.5);
semilogy(indices, ratio, '-', 'LineWidth', 1.5);
grid off; axis square;
ylim([y_min_satisfied, y_max_satisfied]);
xlabel('Index, $i$', 'Interpreter', 'latex', 'FontSize', 16);
ylabel('Magnitude', 'Interpreter', 'latex', 'FontSize', 16);
legend('$\sigma_i$ - Singular values', ...
       '$|c_i|$ - Fourier coefficients', ...
       '$|c_i|/\sigma_i$ - Decay rate', ...
       'Location', 'NorthWest', 'Interpreter', 'latex', 'FontSize', 14);
exportgraphics(gcf, 'dpc_satisfied_picard.png', 'Resolution', 300);
figure('Visible', 'on', 'Units', 'pixels', 'Position', [100 100 600 500]);
loglog(rho, eta, '-', 'LineWidth', 2);
grid off; axis square;
xlabel('$\log \| \mathbf{A}\mathbf{x}_{\lambda} - \mathbf{b} \|_2$', '
    Interpreter', 'latex', 'FontSize', 16);
ylabel('$\log \| \mathbf{x}_{\lambda} \|_2$', 'Interpreter', 'latex', 'FontSize'
    , 16);
exportgraphics(gcf, 'dpc_satisfied_lcurve.png', 'Resolution', 300);

% DPC Not Satisfied
randn('state', 1);
```

```
b_noise = randn(n, 1);
b = b_noise / norm(b_noise);
c = U' * b;
ratio = abs(c) ./ s;
rho = zeros(1, 100);
eta = zeros(1, 100);
for k = 1:100
    lambda = lambdas(k);
    f = s.^2 ./ (s.^2 + lambda);
    rho(k) = norm( (lambda ./ (s.^2 + lambda)) .* c );
    eta(k) = norm( (f .* c) ./ s );
end

% Plotting
figure('Visible', 'on', 'Units', 'pixels', 'Position', [100 100 600 500]);
semilogy(indices, s, '-', 'LineWidth', 2); hold on;
semilogy(indices, abs(c), 'r.', 'MarkerSize', 10);
semilogy(indices, ratio, '-', 'LineWidth', 1.5);
grid off; axis square;
ylim([y_min_unsatisfied, y_max_unsatisfied]);
xlabel('Index, $i$', 'Interpreter', 'latex', 'FontSize', 16);
ylabel('Magnitude', 'Interpreter', 'latex', 'FontSize', 16);
legend('$\sigma_i$ - Singular values', ...
       '$|c_i|$ - Fourier coefficients', ...
       '$|c_i|/\sigma_i$ - Decay rate', ...
       'Location', 'NorthWest', 'Interpreter', 'latex', 'FontSize', 14);
exportgraphics(gcf, 'dpc_unsatisfied_picard.png', 'Resolution', 300);
figure('Visible', 'on', 'Units', 'pixels', 'Position', [100 100 600 500]);
loglog(rho, eta, '-', 'LineWidth', 2);
grid off; axis square;
xlabel('$\log \| \mathbf{A}\mathbf{x}_{\lambda} - \mathbf{b} \|_2$', '
    Interpreter', 'latex', 'FontSize', 16);
ylabel('$\log \| \mathbf{x}_{\lambda} \|_2$', 'Interpreter', 'latex', 'FontSize'
    , 16);
exportgraphics(gcf, 'dpc_unsatisfied_lcurve.png', 'Resolution', 300);
```

Listing 2: MATLAB script for DPC verification on the `shaw` problem.

D GCV plot

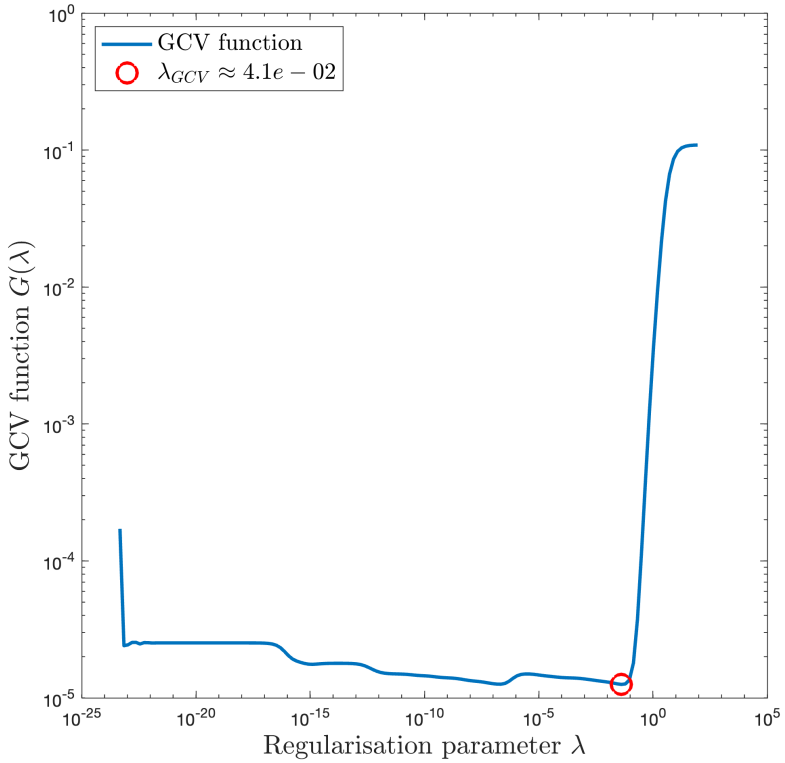


Figure 5: Experimental verification of the statistical GCV parameter-choice method.

E MATLAB code for GCV verification

The following MATLAB script was developed to manually generate the GCV plot in Figure 5 which was used to identify the regularisation parameter λ_{GCV}.

```
% Problem Definition
n = 50;
[A, b_true, x_true] = shaw(n);


noise_level = 0.01;
randn('state', 0);
e = randn(n, 1);
e = e / norm(e) * noise_level * norm(b_true);
b = b_true + e;

% Compute SVD
[U, s, V] = csvd(A);
beta = U'*b;

% Define Lambda Range
lambda_start = 0.1 * s(end)^2;
lambda_end = 10 * s(1)^2;
lambdas = logspace(log10(lambda_start), log10(lambda_end), 200);

n_lambda = length(lambdas);
G = zeros(n_lambda, 1);

m = n;

for i = 1:n_lambda
    lam = lambdas(i);

    f = s.^2 ./ (s.^2 + lam^2);

    res_norm_sq = norm( (1-f) .* beta )^2;

    trace_term = sum(f);
    denominator = (m - trace_term)^2;

    G(i) = res_norm_sq / denominator;
end

% Find Minimum Point
[min_G, idx_min] = min(G);
opt_lambda_gcv = lambdas(idx_min);

% Plotting
figure('Visible', 'on', 'Units', 'pixels', 'Position', [100 100 600 500]);
loglog(lambdas, G, '-', 'LineWidth', 2); hold on;
loglog(opt_lambda_gcv, min_G, 'ro', 'MarkerSize', 12, 'LineWidth', 2);

grid off; axis square;
xlabel('Regularisation parameter $\lambda$', 'Interpreter', 'latex', 'FontSize',
    16);
ylabel('GCV function $G(\lambda)$', 'Interpreter', 'latex', 'FontSize', 16);
legend('GCV function', ['$\lambda_{GCV} \approx ' num2str(opt_lambda_gcv, '%.1e'
    ) '$'], ...
       'Location', 'NorthWest', 'Interpreter', 'latex', 'FontSize', 14);

exportgraphics(gcf, 'gcv_plot.png', 'Resolution', 300);
```

Listing 3: MATLAB script for GCV verification on the `shaw` problem.